

Módulo 4: Datos Sensibles y Secretos

T11: Almacenamiento Seguro de Contraseñas

Carlos Rodríguez Contreras · GitHub: [karrocon](#)

 *De plain text a bcrypt — cómo almacenar contraseñas en la base de datos*

Objetivos

- Entender por qué MD5/SHA1 no son aptos para contraseñas
- Explicar el concepto de salting y work factor en bcrypt/Argon2
- Demostrar la vulnerabilidad de VulnApp (contraseñas en plain text o MD5)
- Migrar el almacenamiento de contraseñas a bcrypt y verificar timing-safe comparison

La evolución del almacenamiento de contraseñas

Plain text → MD5 → SHA-256 → bcrypt → Argon2



Método	Problema
Plain text	Legible directamente en la BD
MD5	Rápido → crackeable con rainbow tables en segundos
SHA-256	Rápido → fuerza bruta con GPU (billones/segundo)
bcrypt	Lento por diseño (work factor configurable) ✓
Argon2	Resistente a GPU + configura memoria ✓✓

VulnApp: contraseñas en texto plano

```
// src/db/database.ts - línea 57
database.prepare(`
  INSERT INTO users (username, password, role, email) VALUES
    ('alice', 'password123', 'admin', 'alice@example.com'),
    ('bob', 'qwerty', 'user', 'bob@example.com'),
    ('carol', 'letmein', 'user', 'carol@example.com')
`).run();
```

```
-- Cualquiera con acceso a la BD ve:
SELECT username, password FROM users;
-- alice | password123
-- bob   | qwerty
```

💀 Si hay una brecha de BD (SQLi, backup filtrado), **todas las contraseñas expuestas.**

¿Por qué bcrypt y no SHA-256?

SHA-256: diseñado para ser RÁPIDO

GPU moderna: ~10 BILLONES de SHA-256/segundo
Contraseña de 8 chars: crackeada en horas

bcrypt: diseñado para ser LENTO

Work factor 12: ~0.3 segundos por hash
10 billones de intentos tardarían: ~95.000 AÑOS

✅ El "work factor" (cost) se puede aumentar conforme mejora el hardware. bcrypt(12) hoy → bcrypt(14) en 5 años.

¿Qué es salting?

```
Sin salt: hash("password123") = siempre el mismo hash
          → Rainbow table: pre-computar hashes de millones de passwords
```

Con salt: `hash("password123" + "x7k9m2")` = hash único por usuario
→ Rainbow tables inútiles

bcrypt incluye salt automáticamente:

```
$2b$12$LJ3m4ov3rExAmPlEsAlTrandomHash...
|   |   |   Salt (22 chars)   _____
|   |   |   Work factor (2^12 = 4096 iteraciones)
|   |   |   Versión bcrypt
```

El fix: migrar a bcrypt

```
import bcrypt from 'bcryptjs';

// Al REGISTRAR:
const hash = await bcrypt.hash(password, 12);
db.prepare('INSERT INTO users (username, password) VALUES (?, ?)').run(username, hash);

// Al hacer LOGIN:
const user = db.prepare('SELECT * FROM users WHERE username = ?').get(username);
const valid = await bcrypt.compare(inputPassword, user.password);
if (!valid) return res.status(401).json({ error: 'Credenciales incorrectas' });
```

⚠ **Nunca compares hashes con `===` — usa `bcrypt.compare()` que es timing-safe.**

Timing attacks — por qué importa la comparación segura

```
// ❌ VULNERABLE a timing attack
if (storedHash === inputHash) { ... }
// La comparación cortocircuita en el primer byte diferente
// → se puede deducir el hash byte a byte midiendo tiempos

// ✅ SEGURO - tiempo constante
import { timingSafeEqual } from 'crypto';
timingSafeEqual(Buffer.from(a), Buffer.from(b));
// bcrypt.compare() ya lo hace internamente
```

Políticas de contraseñas

Regla	Recomendación (NIST 800-63B)
Longitud mínima	8 caracteres (mejor 12+)
Complejidad forzada	✗ No recomendada (genera <code>P@ssw0rd!</code>)
Blacklist	✓ Rechazar las 10.000 más comunes
Rotación periódica	✗ No forzar (genera patrones predecibles)
MFA	✓ Siempre que sea posible
Indicador de fuerza	✓ Feedback visual en tiempo real

Argon2 vs bcrypt – comparativa moderna

Aspecto	bcrypt	Argon2id
Año	1999	2015 (ganador PHC)
Resistencia GPU	Buena	Excelente (memory-hard)
Configuración	Work factor (cost)	Time, memory, parallelism
Máximo password	72 bytes	Sin límite
Recomendado por	Muchos frameworks	OWASP, NIST

```
// Argon2 con node (argon2 package)
import argon2 from 'argon2';

const hash = await argon2.hash(password, {
  type: argon2.argon2id,      // híbrido: resiste side-channel + GPU
  memoryCost: 65536,          // 64MB RAM
  timeCost: 3,                 // 3 iteraciones
  parallelism: 4               // 4 threads
});

const valid = await argon2.verify(hash, inputPassword);
```

✓ Si empiezas un proyecto nuevo, usa **Argon2id**. Si ya usas bcrypt con cost ≥ 12 , está bien mantenerlo.

Lab T11: Migrar contraseñas a bcrypt

Objetivo: sustituir el almacenamiento inseguro de contraseñas por bcrypt

Setup: `cd VulnApp && npm start`

1. Inspecciona la tabla `users` en la BD: `SELECT * FROM users`
2. Observa que las contraseñas están en plain text (o MD5)
3. Instala `bcryptjs` y actualiza `src/routes/auth.ts` :
 - registro: `bcrypt.hash(password, 12)`
 - login: `bcrypt.compare(input, stored)`
4. Registra un nuevo usuario y verifica que la contraseña está hasheada en la BD